

Tema 5: Integridad de mensajes, firmas y certificados

Docente : Héctor Xavier Limón Riaño
email: xavier120@hotmail.com

MAC

- ▶ El MAC protege la integridad del mensaje.
- ▶ Un atacante sin la llave no debe poder alterar el mensaje sin que se pueda detectar
- ▶ Mientras la llave se mantenga privada, el MAC provee autenticidad: el receptor sabe que sólo el emisor real pudo enviar el mensaje porque sólo así puede generarse un MAC correcto

HMAC

HMAC

En el RFC el tamaño de bloque de H se define como B , éstas son las reglas de restricción de K :

1. Si K es más corto que B , entonces a K se le tienen que agregar ceros de padding hasta ser de longitud B
2. Si K es más largo que B , entonces K se reduce aplicando H a K , osea $H(K)$
3. Si se aplicó 2 y la llave quedó muy corta, entonces se aplica 1

HMAC

- ▶ Si se aplican las reglas anteriores, siempre se va a tener una llave de longitud B
- ▶ En el RFC *ipad* se define como el byte $0x36$ repetido B veces
- ▶ *opad* es el byte $0x5c$ repetido B veces
- ▶ Los pads parecen arbitrarios pero tienen un sentido de seguridad matemático que ayuda a mitigar problemas si los algoritmos de hash son vulnerados (como con MD5 o SHA-1)

HMAC

- ▶ Luego se computa $K \oplus opad$
- ▶ Para obtener el outer-hash final se concatena el resultado de arriba con el resultado del inner-hash y se aplica H
- ▶ El outer-hash es el HMAC de *text* keyed con K

Ejercicio

- ▶ Realiza una implementación manual de HMAC para SHA-256 de acuerdo a la explicación del RFC
- ▶ De acuerdo al tiempo que quede de clase el profesor podría implementar parte del ejercicio

HMAC

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import hashes, hmac
3
4 key = b"CorrectHorseBatteryStaple"
5 h = hmac.HMAC(key, hashes.SHA256(), backend=default_backend())
6 h.update(b"hello world")
7 h.finalize().hex() # HMAC en hexadecimal
```

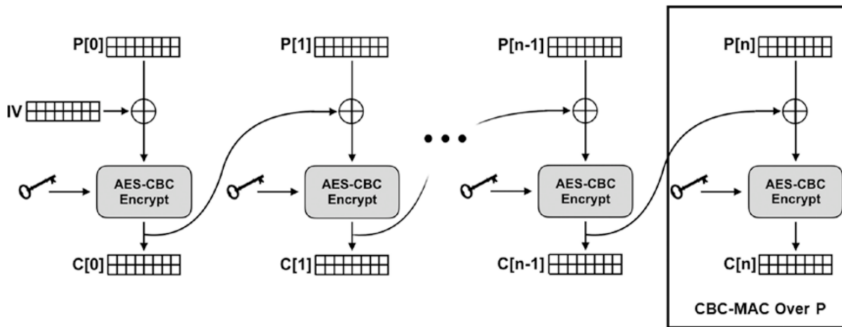
CBC-MAC

- ▶ Similar a AES CBC
- ▶ Recordar que CBC es Cipher Block Chaining
- ▶ Un MAC a menudo es llamado un **tag** (etiqueta) del mensaje
- ▶ Es como una etiqueta que se agrega a un regalo: es información extra que se asocia al contenido principal

CBC-MAC

- ▶ En el caso de AES, recuerda que se estaba limitado a cifrar 128 bits de bloque a la vez
- ▶ La idea de AES CBC es encadenar la salida de un bloque cifrado con el siguiente, para que no haya independencia entre bloques y sea más complejo alterar la integridad de los datos
- ▶ Si se alterara un bit en el primer bloque ese cambio crea una cascada de cambios en los bloques siguientes hasta llegar al último bloque
- ▶ Este efecto hace que el último bloque de CBC sea por si mismo un MAC

CBC-MAC



CBC-MAC

- ▶ En esencia CBC-MAC es AES CBC, recuperando el último bloque
- ▶ Pero se deben tomar en cuenta varias consideraciones de seguridad
- ▶ Veamos una implementación no segura

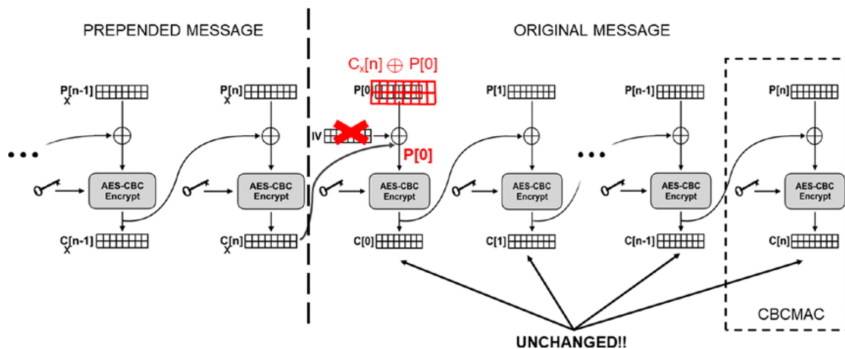
CBC-MAC: ataque antepuesto

- ▶ MAC prepend attack
- ▶ Este ataque funciona con la implementación insegura antes vista
- ▶ La idea básica es que si se tienen dos mensajes M_1 y M_2 y sus tags correspondientes t_1 y t_2 generados con la misma llave k , es posible generar un mensaje nuevo manipulado M_3 cuyo tag t_3 sea igual a t_1 o t_2
- ▶ Osea que es posible mandar un mensaje manipulado y hacer que parezca que es el mensaje M_1 o M_2

CBC-MAC: ataque antepuesto

- ▶ Básicamente lo que hace el ataque es remplazar el primer bloque de M_2 por uno manipulado
- ▶ La víctima verá todo el mensaje M_1 completo (posiblemente con su padding), un bloque que se verá posiblemente corrupto, y el mensaje M_2 sin su primer bloque

CBC-MAC: ataque antepuesto



CBC-MAC: ataque antepuesto

- ▶ Existen muchas soluciones a este ataque
- ▶ Una de ellas es por ejemplo agregar siempre la longitud del mensaje al inicio de éste, de esta manera puede revisarse si se agregó información extra
- ▶ Para hacer una implementación segura se requiere además:
 - ▶ Si además los datos se están cifrando con AES-CBC, no se debe usar la misma llave para cifrar y obtener el MAC
 - ▶ El IV debe ser diferente de cero
- ▶ Lo mejor es utilizar una versión segura estable, por ejemplo CMAC de la biblioteca cryptography que es una versión mejorada de CBC-MAC descrita en el RFC 4493

CBC-MAC

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import cmac
3 from cryptography.hazmat.primitives.ciphers import algorithms
4
5 c = cmac.CMAC(algorithms.AES(key), backend=default_backend())
6 c.update(b"message to authenticate")
7 mac = c.finalize()
```

HMAC y CBC-MAC

- ▶ Tanto HMAC como CBC-MAC son buenas opciones para un algoritmo MAC
- ▶ HMAC puede ser más eficiente en la mayoría de sistemas, con excepción de aquellos que tienen hardware especializado para AES
- ▶ Ante la duda, utiliza HMAC

Cifrado y MAC

- ▶ En muchas circunstancias un mensaje necesita ser cifrado y protegido contra modificaciones
- ▶ Si lo analizan, en lo que se vio anteriormente no se especifica si los mensajes para los cuales se derivaba el MAC estaban cifrados o no, en realidad podría ser cualquiera de las dos, los métodos funcionarían igual
- ▶ Pero, si se va a cifrar ¿En qué momento hay que derivar el MAC? ¿Antes o después de cifrar?

Cifrado y MAC

- ▶ Lo que recomienda la mayoría de los criptógrafos (aunque no todos), es utilizar cifra luego aplica MAC (Encrypt-Then-MAC)
- ▶ Algunas vulnerabilidades han sido encontradas para aplica MAC luego cifra (MAC-Then-Encrypt), de hecho por eso ya no se usa en TLS. Por ejemplo el ataque de padding oráculo a CBC sólo funciona con este esquema
- ▶ Cifra luego aplica MAC parece contra-intuitivo, puesto que lo que queremos saber es si los datos originales (texto plano) llegaron bien

Criptografía asimétrica

- ▶ Se tienen dos llaves: una pública y privada
- ▶ La llave pública se puede derivar de la privada, pero no al revés
- ▶ Si se cifra con la llave pública sólo se puede descifrar con la llave privada y vice-versa
- ▶ La llave privada siempre debe permanecer privada, mientras que la pública está pensada para ser compartida

Criptografía asimétrica

- ▶ ¿Cómo sabes al recibir una llave pública que proviene de la entidad oficial que esperas?, un atacante podría darte su llave pública para engañarte (más sobre este problema más adelante)
- ▶ El cifrado asimétrico es ineficiente, muy ineficiente para documentos largos
- ▶ Descifrar un documento largo sólo para saber si proviene de la entidad correcta no es viable
- ▶ En el caso de RSA (y muchos otros cifradores asimétricos), no tiene forma de revisar la integridad de los datos, además de que sólo puede procesar documentos tan grandes como el tamaño de módulo (2048 bits tradicionalmente)

Curvas elípticas

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import hashes
3 from cryptography.hazmat.primitives.asymmetric import ec
4
5 private_key = ec.generate_private_key(
6     ec.SECP384R1(), default_backend())
7 public_key = private_key.public_key()
8
9 data = b"this is some data I'd like to sign"
10 signature = private_key.sign(data,
11     ec.ECDSA(hashes.SHA256()))
12
13 public_key.verify(signature, data, ec.ECDSA(hashes.SHA256()))
```


Certificados

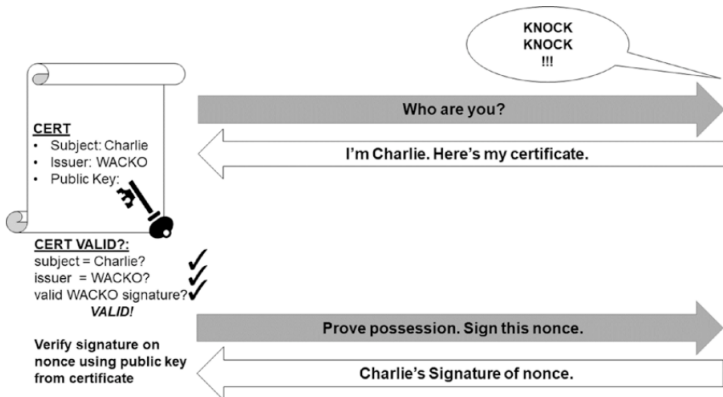
- ▶ Los metadatos incluyen información como la identidad del propietario de la llave, la identidad del issuer, fecha de expiración, número de serie, entre otros
- ▶ La idea es ligar los metadatos, especialmente los de identidad, a la llave pública
- ▶ La identidad puede ser un nombre, email, una URL, u otro identificador acordado



Certificados

- ▶ Existe un problema con el modelo visto hasta ahora
- ▶ Imagina que un compañero al que no conoces que se hace llamar Charlie llega y te presenta su certificado, firmado por el profesor en el que confías
- ▶ Podrás, mediante la llave pública del maestro, la cual ya tenías, verificar la firma y sacar la información de identidad, que diga que es de Charlie
- ▶ Pero, ¿Cómo puedes saber que la persona que está ahí realmente es Charlie?
- ▶ Las llaves públicas cualquiera las tiene, alguien pudo haber generado un certificado falso engañando a la entidad de confianza

Certificados



Certificados y confianza

- ▶ El tener un certificado raíz confiable también facilita las cosas
- ▶ Sólo se necesita tener el certificado raíz, cualquier cadena de certificados generados a partir de ese certificado raíz puede ser verificado siguiendo la cadena en reversa

Prevención

- ▶ El seguimiento de nonces puede ser problemático si existen miles o millones de mensajes
- ▶ ¿Cuál es la mejor estrategia? ¿Guardar los últimos N nonces? ¿Guardar cada nonce sólo por N minutos?
- ▶ Si eliges un patrón de comportamiento y el atacante logra determinarlo, entonces puede burlar la seguridad
- ▶ Por ejemplo, si decides guardar el nonce por 5 minutos y el atacante se da cuenta de esto, podría mandar mensajes que se enviaron hace 6 minutos

Prevención

- ▶ Una mejor solución es combinar nonces aleatorios y timestamps
- ▶ El timestamp ayuda a identificar mensajes realmente viejos
- ▶ Mientras que el nonce ayuda a identificar repeticiones dentro de la ventana de tiempo permitida
- ▶ Obviamente los nonces deben mantenerse el mismo tiempo que la ventana y no es necesario mantenerlos más tiempo, lo que ayuda a su manejo
- ▶ Esta aproximación también relaja los problemas de sincronización de relojes entre sistemas, pues no se necesita tanta exactitud